

一、设计模式基础认知

1. 模式起源与基本定义

设计模式最早起源于建筑业，后被引入软件工程领域。

经典著作：GoF（四人组）《设计模式：可复用面向对象软件的基础》，全书共收录 23 种经典面向对象设计模式。

核心定义：设计模式是软件开发领域经过验证、可复用的通用设计经验与解决方案，用于解决项目中重复出现的设计问题，它是设计思想，而非固定代码。 (第二章第 13 题)

2. 设计模式三大分类（按使用目的划分）

创建型模式：核心关注对象的创建过程，隔离对象创建与对象使用逻辑。(第二章第 13 题)

结构型模式：核心关注类与对象的组合关系，构建灵活、稳定的程序结构。(第五章第 14 题)

行为型模式：核心关注对象间交互、职责分配与算法流程控制。(第六章第 10 题、第六章第 20 题、第六章第 28 题)

3. 设计模式的优点

提升代码复用性、可读性、可维护性与系统扩展性；统一团队设计思维，降低沟通成本。但值得注意的是，设计模式无法彻底消除程序 Bug；并非所有场景都需要使用，过度使用会增加代码复杂度。

二、面向对象六大设计原则

1. 单一职责原则

核心内容：一个类仅有一个引起它变化的原因，一个类只负责一项核心职责。

作用：降低类之间耦合度，类修改影响范围小，便于后期维护与代码复用。

2. 开闭原则（设计模式最核心原则）

核心内容：软件实体对扩展开放，对修改关闭。新增功能优先新增代码，尽量不修改原有成熟代码。

应用场景：绝大多数工厂模式、观察者模式、策略模式均遵循该原则。(第二章第 6 题)

易错点：简单工厂模式违背开闭原则。(第二章第 7 题、第二章第 15 题)

3. 里氏代换原则

核心内容：在程序中，子类对象可以完全替换父类对象，替换后原有业务逻辑不受影响。

典型反例：将“正方形”设计为“长方形”的子类，违背本原则。

基本要求：子类不能重写、破坏父类已实现的核心行为。

4. 依赖倒转原则

核心内容：高层模块与低层模块统一依赖抽象，不依赖具体实现；倡导面向接口编程，而非面向实现编程。

落地方式：优先使用接口、抽象类，弱化具体类之间的直接依赖。

5.接口隔离原则

核心内容：拆分臃肿的大接口，使用多个细粒度专用接口；客户端只依赖自身需要的接口。
作用：避免类被迫实现无用方法，减少代码冗余。

6.迪米特法则（最少知道原则）

核心内容：一个对象只与直接朋友通信，尽量少了解其他对象的内部细节。
作用：有效降低类与类之间的耦合度。
易错点：本原则目的是解耦，而非增强耦合。

7.合成复用原则

核心内容：代码复用优先使用组合/关联关系，其次考虑继承。
原因：继承属于强耦合关系，组合、关联灵活性更高。

三、UML 类图核心关系

1.五大关系（按耦合强度：由强 → 弱排序）

组合 > 聚合 > 关联 > 泛化/实现 > 依赖

2.组合（强整体-部分关系）

图形符号：带实心菱形的实线
特点：部分对象无法脱离整体独立存在，生命周期与整体绑定。

3.聚合（弱整体-部分关系）

图形符号：带空心菱形的实线
特点：部分对象可以独立存在，整体与部分生命周期互不影响。

4.关联关系

图形符号：普通实线
特点：类之间长期、稳定的引用关系。

5.泛化 / 实现

泛化（类继承）：带空心三角形的实线
实现（类实现接口）：带空心三角形的虚线
特点：表示父子类、类与接口之间的从属关系。

6.依赖关系

图形符号：带箭头的虚线
特点：临时、偶然的调用关系，常见于方法参数、局部变量、静态方法调用。

四、创建型设计模式

核心作用：封装对象创建逻辑，将对象创建与对象使用分离，统一对象创建规则。

1.单例模式

核心目标：保证一个类在整个系统中仅有一个实例，并提供全局统一访问入口。[\(第三章第1题\)](#)

实现要点：构造方法私有化、私有静态实例对象、公共静态获取实例方法。(第三章第 3 题、第三章第 17 题)

两种主流实现：

1) 饿汉式：类加载时直接创建实例，天然线程安全，提前占用系统资源。(第三章第 5 题)

2) 懒汉式：首次调用方法时才创建实例（延迟加载），无同步控制时多线程不安全。(第三章第 10 题)

适用场景：日志工具、打印服务、全局配置类、资源管理器。(第三章第 18 题)

易错点：单例类构造方法必须私有化，禁止外部通过 `new` 创建对象。

线程安全实现：双重检查锁定配合 `volatile` 可解决懒汉式线程安全问题。(第三章第 6 题、第三章第 11 题)

2. 简单工厂模式

结构组成：抽象产品、具体产品、单一工厂类。(第二章第 10 题、第二章第 16 题)

工作逻辑：工厂类根据传入参数，动态创建不同类型的具体产品对象。(第二章第 1 题、第二章第 4 题、第二章第 18 题)

优缺点：使用简单；新增产品必须修改工厂类代码，违背开闭原则。(第二章第 3 题、第二章第 7 题、第二章第 11 题、第二章第 15 题)

3. 工厂方法模式

结构组成：抽象产品、具体产品、抽象工厂、多个具体工厂子类。(第二章第 8 题、第二章第 12 题、第二章第 17 题)

工作逻辑：将对象实例化逻辑延迟到工厂子类中完成。(第二章第 2 题、第二章第 5 题、第二章第 9 题)

优缺点：符合开闭原则，新增产品只需新增对应工厂子类；缺点是类数量增多，结构复杂度上升。(第二章第 6 题、第二章第 14 题)

4. 抽象工厂模式

基础概念：

1) 产品等级结构：同一品类的一系列产品。

2) 产品族：不同等级、功能相互关联的一组产品。

核心特点：专门用于生产一组相关联的产品族。

开闭原则倾斜性：新增产品族符合开闭原则；新增产品等级结构违背开闭原则。

5 补充模式（了解即可）

原型模式：通过复制已有对象来创建新对象。

建造者模式：分步构建复杂对象，分离对象构建与对象表示。

6. 三大工厂模式对比

简单工厂：单个工厂类，适合产品种类少、变动小的场景，违背开闭原则。

工厂方法：多个工厂子类，面向单一产品等级，符合开闭原则。

抽象工厂：多个工厂子类，面向产品族，适合多系列关联产品场景。

五、结构型设计模式

核心作用：灵活组合类与对象，解决接口适配、访问控制、结构搭建等问题。

1.适配器模式

核心作用：转换接口，解决接口不兼容问题，让原本无法协作的类正常工作。(第三章第 2 题)

两种实现方式：

1) 对象适配器：基于组合/关联实现，兼容性强，主流使用方式。(第三章第 8 题、第三章第 12 题)

2) 类适配器：基于继承实现，受编程语言限制，Java 中极少使用。(第三章第 4 题) 角色组成：目标接口、适配者（原有类）、适配器。(第三章第 15 题)

注意：适配器仅做接口转换，不修改原有类的业务逻辑。(第三章第 16 题、第三章第 19 题、第三章第 20 题)

2.外观模式（门面模式）

核心思想：为多个复杂子系统提供统一高层访问入口，简化客户端调用。(第四章第 5 题)

结构组成：外观类、多个独立子系统类。(第四章第 3 题、第四章第 4 题、第四章第 17 题)

特点：客户端仅与外观类交互，不直接访问底层子系统，降低使用复杂度。(第四章第 2 题、第四章第 7 题)

适用场景：智能家居系统、第三方复杂接口封装、多层业务系统调用。(第四章第 12 题、第四章第 14 题、第四章第 15 题)

3.代理模式

核心思想：通过代理对象访问真实目标对象，实现访问控制、功能增强。(第五章第 7 题)

结构组成：抽象主题、真实主题（目标类）、代理类。(第五章第 5 题)

特点：代理类与真实类实现同一接口；可在目标方法执行前后添加额外逻辑。(第五章第 28 题)

典型场景：权限控制、远程访问、程序快捷方式。(第五章第 1 题、第五章第 4 题、第五章第 10 题、第五章第 13 题)

代理模式常见类型：远程代理、虚拟代理、保护代理、缓冲代理、智能引用代理。(第五章第 17 题)

4.组合模式

核心思想：将对象组织为树形结构，实现客户端统一处理单个对象与组合对象。(第四章第 1 题、第四章第 9 题)

适用场景：文件系统、组织架构、多级菜单等树形业务场景。(第四章第 6 题、第四章第 11 题、第四章第 13 题)

5.补充了解

桥接模式：分离抽象与实现，让抽象和实现可以独立变化；

装饰模式：动态为对象拓展功能。

六、行为型设计模式

核心作用：规范对象之间的交互、通信、算法流转与职责分配。

1.观察者模式（发布-订阅模式）

核心思想：定义对象间一对多依赖关系；当目标对象状态改变时，自动通知所有已注册的观察者。(第六章第 19 题、第六章第 37 题)

角色组成：抽象主题、具体主题（被观察者）、抽象观察者、具体观察者。(第六章第 17 题、第六章第 27 题)

特点：目标与观察者采用抽象耦合，符合依赖倒转原则；一个目标可对应多个观察者。(第六章第 25 题、第六章第 30 题、第六章第 37 题)

JDK 中的实现：Observable 类和 Observer 接口。(第六章第 2 题)

Java 事件处理机制底层采用了观察者模式思想。(第六章第 35 题)

2.迭代器模式

核心思想：提供统一方式遍历聚合对象，不暴露对象内部存储结构。(第六章第 1 题、第六章第 18 题)

角色组成：抽象聚合、具体聚合、抽象迭代器、具体迭代器。(第六章第 4 题、第六章第 15 题、第六章第 23 题)

设计思想：分离数据存储与数据遍历，符合单一职责原则。(第六章第 33 题)

Java 内置迭代器位于 java.util 包。(第六章第 5 题)

3.命令模式

核心思想：将请求封装为对象，彻底解耦请求发送者与请求接收者。(第五章第 15 题、第五章第 30 题)

拓展能力：支持请求排队、操作撤销、请求日志、宏命令等功能。(第五章第 3 题、第五章第 11 题、第五章第 12 题、第五章第 18 题)

角色组成：抽象命令、具体命令、接收者、调用者。(第五章第 2 题、第五章第 6 题)

别名：动作模式、事务模式。(第五章第 22 题)

4.策略模式

核心思想：封装多种算法，支持运行时动态切换算法。(第六章第 6 题、第六章第 9 题、第六章第 36 题)

角色组成：上下文、抽象策略、具体策略。(第六章第 3 题、第六章第 26 题)

策略模式适用场景：替代多重 if-else 条件语句、封装算法保证保密性、动态选择多种算法。(第六章第 22 题、第六章第 38 题)

5.补充了解

职责链模式：请求沿对象链路传递，由多个对象依次处理请求。